CAMPUSHIRE AI

Intelligent Placement and Skill-Matching Web Application

SUBMITTED BY

Riddhi Sharma • Sai Prasad Neelmani • Saloni Kumari
Shubham Purohit • Toyeshh Somvansi

PROJECT GUIDE

Er. Rambabu Buri

Academic Year 2026–2027

Problem Statement

Why a unified placement workflow is needed

- Student profiles, skills and vacancy information are scattered across spreadsheets, e-mail and messaging groups.
- Duplicate or outdated records make it difficult to track the status of each application.
- Students cannot easily see which required skills they lack for a given role.
- Eligibility criteria and shortlisting can be applied inconsistently, with limited audit trail.
- Placement officers need one consistent record of students, jobs and applications.

Aim and Objectives

- Provide registration and login for Student, Recruiter and Administrator roles.
- Store student profiles and skills in a normalised relational database.
- Allow recruiters and administrators to create and manage job postings.
- Allow students to browse vacancies and apply; prevent duplicate applications.
- Calculate an explainable skill-coverage score with matched and missing skills.
- Protect data through hashing, validation, authorisation and prepared statements.
- Verify the main workflows through ten documented test cases.

Proposed System and User Roles

CampusHire AI combines profile management, job management and explainable matching

STUDENT

Maintains profile and skills, browses open jobs, views match score and applies.

RECRUITER

Publishes jobs with required skills and reviews applications for own vacancies.

ADMINISTRATOR

Manages users and job records; oversees platform activity.

Guests may view public pages only. All privileged actions are authorised on the server.

System Modules

1. AUTHENTICATION

Registration, login, role-based access, session handling.

2. STUDENT PROFILE

Academic details, skills, projects and certifications.

3. JOB MANAGEMENT

Job postings, eligibility criteria, required skills, deadlines.

4. APPLICATIONS

Apply to open jobs; one application per student per job; status tracking.

5. SKILL MATCHING

Coverage score with matched and missing skills.

6. ADMINISTRATION

User and job oversight by authorised roles.

Technology Stack

Java web application packaged as a Maven WAR

PRESENTATION

JSP, JSTL, HTML, CSS, JavaScript

APPLICATION

Java 17, Servlets (Servlet 4.0, javax namespace), service and DAO classes, JDBC

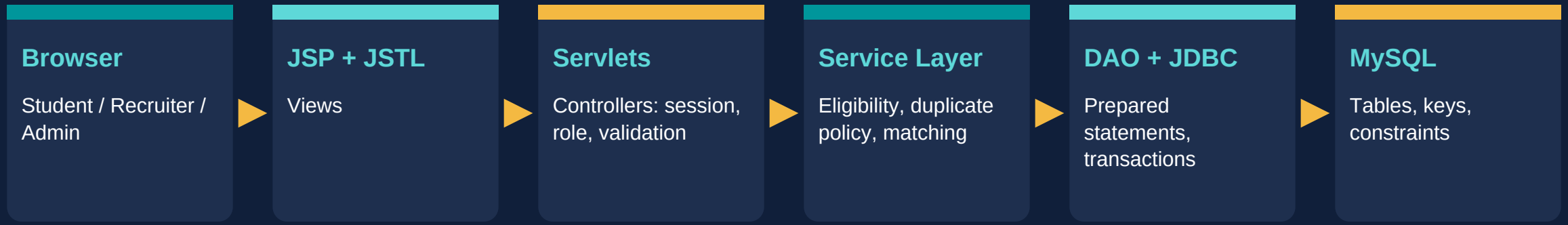
DATA AND SERVER

MySQL (InnoDB) with Connector/J; Apache Tomcat 9; Apache Maven; Eclipse IDE

Tomcat 9 uses the javax.servlet namespace; Tomcat 10+ would require jakarta.servlet.

System Architecture

Layered MVC design with clear separation of responsibilities



Request flows left to right; the response returns through the controller to a JSP view.

Database Design

Normalised schema; integrity enforced by the database

Table	Purpose
users	Accounts: email (unique), password hash, role
student_profiles / recruiter_profiles	Role-specific details linked one-to-one to users
jobs	Vacancies with status, deadline and minimum CGPA
skills, student_skills, job_skills	Skill catalogue and many-to-many mappings
applications	Student–job applications with status; UNIQUE (student_id, job_id)

Foreign keys prevent orphaned records; the unique constraint blocks duplicate applications even under concurrent requests.

Skill-Matching Method

Transparent rule-based coverage score — not a trained machine-learning model

Coverage score = 100 × |S ∩ J| / |J|

S = student's normalised skills; J = skills required by the job

ROLE REQUIRES (6)

Java, SQL, HTML, CSS, Git, JDBC

STUDENT HAS

Java, SQL, HTML, CSS

RESULT

Matched: 4 of 6 → **66.7%**
Missing: Git, JDBC

Worked example for illustration. The score supports decisions; it does not replace human review.

Security and Responsible Use

- Passwords stored only as salted bcrypt (or Argon2) hashes; session id regenerated at login.
- All SQL uses PreparedStatement to prevent SQL injection; output escaped to prevent XSS.
- Server-side validation and role/ownership checks on every privileged action.
- Views kept under WEB-INF; database credentials kept outside source control.
- Matching uses no protected attributes; matched and missing skills are always shown.
- The score is decision support only; final placement decisions remain with people.

Verification Plan

Ten test cases; actual results are recorded on the running build

ID	Scenario	Expected result
TC-01	Valid registration	Account created; password stored as hash
TC-02	Duplicate e-mail	Rejected; no second account
TC-03	Invalid login	Generic error; no session
TC-04	Valid login	Dashboard displayed
TC-05	Active job listing	Only open jobs shown
TC-06	Valid application	One row with status SUBMITTED
TC-07	Duplicate application	Rejected; count remains one
TC-08	Skill match (Java, MySQL, Git vs Java, MySQL, REST)	Matched 2, missing REST, 66.7%
TC-09	Student edits a job	HTTP 403 / access denied
TC-10	Job with no required skills	“Match unavailable”; no division by zero

Current Status and Limitations

- The design, schema and application architecture are defined; the test plan is prepared.
- Test cases state expected outcomes; observed results will be recorded after execution on the working build.
- No performance measurements or pass rates are claimed at this stage.
- Matching is a simple coverage score; it ignores skill depth, recency and soft skills.
- Quality of matching depends on consistent skill naming (synonym normalisation is planned).

Future Scope

MATCHING

Synonym normalisation, weighted skills, documented evaluation and fairness review.

FEATURES

Dashboards, resume parsing, e-mail notifications, status history and audit logs.

ENGINEERING

Versioned migrations, query benchmarking, HTTPS, backups and secrets management.

Conclusion

CONNECT

Student profiles, jobs and applications in one workflow.

EXPLAIN

Matched and missing skills, not just a score.

SECURE

Hashed passwords, prepared statements and role-based access.

Thank you • Questions?